

RELATÓRIO DE ENGENHARIA REVERSA

Sistema de Gestão de Chamados

Análise Técnica Detalhada · C# / WinForms / .NET 10
2025

SUMÁRIO

1. Visão Geral do Projeto
2. Arquitetura da Solução
3. Tecnologias Utilizadas
4. Análise Detalhada do Código
5. Fluxo de Funcionamento
6. Regras de Negócio Identificadas
7. Banco de Dados (Persistência)
8. Segurança
9. Padrões de Projeto e Melhorias
10. Resumo para Apresentação

1 VISÃO GERAL DO PROJETO

O sistema tem como objetivo **automatizar e organizar** a abertura e o acompanhamento de chamados técnicos (tickets), conectando clientes que possuem problemas a atendentes responsáveis por resolvê-los.

Problema Resolvido

Substitui o controle manual ou informal de solicitações de suporte, permitindo:

- Registro centralizado de clientes e atendentes.
- Rastreamento do status de cada chamado: **Aberto, Em Andamento, Resolvido**.
- Visão geral rápida da carga de trabalho por meio de um Dashboard.

Público-alvo

Departamentos de TI, equipes de suporte técnico ou pequenas empresas de assistência técnica.

Principais Funcionalidades

- **Gestão de Clientes:** Cadastro e listagem de clientes.
- **Gestão de Atendentes:** Cadastro e listagem de profissionais de suporte.
- **Controle de Chamados:** Abertura vinculando cliente e atendente, alteração de status e filtragem.
- **Dashboard:** Painel com contadores em tempo real de clientes, atendentes e status de chamados.

2 ARQUITETURA DA SOLUÇÃO

O projeto segue uma organização baseada em **separação de responsabilidades**, com quatro camadas bem definidas.

Estrutura de Pastas

```
/ (Raiz)
├── Program.cs → Ponto de entrada da aplicação
├── Models/ → Definições de Entidades (POCOs)
│   ├── Atendente.cs
│   ├── Cliente.cs
│   ├── Chamado.cs
│   └── StatusChamado.cs
├── Services/ → Lógica de Negócio e Persistência
│   ├── AtendenteService.cs
│   ├── ClienteService.cs
│   ├── ChamadoService.cs
│   └── DataPersistence.cs
├── Forms/ → Interface do Usuário (UI)
│   ├── FrmPrincipal.cs
│   ├── FrmAtendentes.cs
│   ├── FrmClientes.cs
│   └── FrmChamados.cs
```

Fluxo de Dependências

UI (Forms) → Services → Models → Persistence (JSON)

A aplicação utiliza um fluxo unidirecional, garantindo que cada camada conheça apenas a camada imediatamente inferior.

Fluxo Geral de Execução

1. Program.cs inicia a aplicação e abre o FrmPrincipal.
2. FrmPrincipal carrega as estatísticas do Dashboard via DataPersistence.
3. O usuário navega para as telas específicas (FrmClientes, FrmAtendentes, FrmChamados).
4. As telas interagem com as classes de Service, que processam a lógica e salvam via DataPersistence.

3 TECNOLOGIAS UTILIZADAS

Tecnologia	Versão / Tipo	Finalidade
C#	13.0 / .NET 10	Linguagem principal de desenvolvimento.
WinForms	.NET 10 Windows	Framework para criação da interface gráfica.
System.Text.Json	Nativo .NET	Serialização e desserialização para arquivos JSON.
JSON	Formato de Arquivo	Armazenamento persistente dos dados.
PowerShell	Shell	Automação de build e execução via CLI.

4 ANÁLISE DETALHADA DO CÓDIGO

A. Camada de Modelos (Models)

As classes de modelo são simples (POCO — Plain Old CLR Objects) e servem para transportar dados entre as camadas.

Cliente

Atributos: Id (int), Nome (string), Contato (string)

Atendente

Atributos: Id (int), Nome (string), Setor (string)

Chamado

Atributos: Id (int), Cliente (Objeto), Atendente (Objeto), Descricao (string), DataAbertura (DateTime), Status (Enum)

StatusChamado

Atributos: Enum: Aberto (1), EmAndamento (2), Resolvido (3)

B. Camada de Serviços (Services)

DataPersistence (Estática)

Motor de banco de dados do sistema. Dois métodos principais:

- **Save<T>(fileName, data):** Converte a lista de objetos para JSON e escreve no arquivo.
- **Load<T>(fileName):** Lê o arquivo JSON e converte de volta para uma lista de objetos.

AtendenteService / ClienteService

Ambas seguem o mesmo padrão de CRUD básico:

- **Cadastrar(...):** Valida se os campos não estão vazios, gera ID incremental e salva no JSON.
- **Listar():** Retorna a lista completa de registros.
- **BuscarPorId(id):** Localiza um registro específico via LINQ (FirstOrDefault).

ChamadoService

A classe mais complexa da camada de negócio:

- **Abrir(cliente, atendente, descricao):** Valida a existência dos objetos e cria novo chamado com status Aberto.
- **AlterarStatus(id, novoStatus):** Localiza o chamado pelo ID e atualiza seu estado.
- **ListarAbertos():** Filtra apenas chamados que não foram resolvidos.

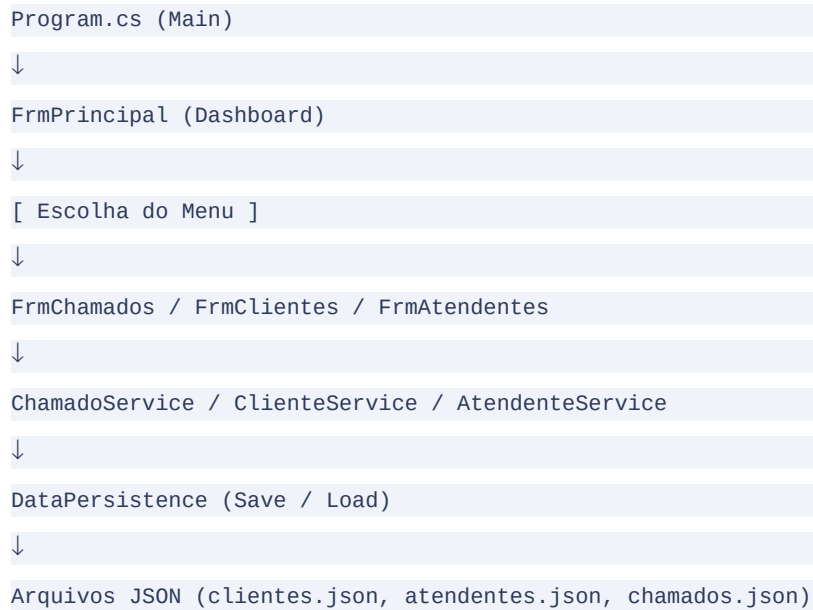
C. Camada de Interface (Forms)

As telas utilizam componentes criados via código (não via Designer), com foco em design moderno — cores Hex, Padding e FlatStyle.

- **FrmPrincipal:** Gerencia a navegação e o Dashboard. Função AtualizarDashboard() recontabiliza os itens a cada abertura de tela.
- **FrmAtendentes / FrmClientes:** Telas de cadastro com DataGridView e campos de texto para entrada.

- **FrmChamados:** Tela mais rica — ComboBoxes vinculados, busca em tempo real via TextChanged, atualização de status via Enum e formatação condicional de cores (vermelho/laranja/verde).

5 FLUXO DE FUNCIONAMENTO



Passo a Passo Detalhado

Inicialização: `Program.cs` instancia o `FrmPrincipal`.

Carga de Dados: O Dashboard chama `DataPersistence` para ler os arquivos JSON e contar os registros.

Ação do Usuário: O usuário clica em 'Clientes' → abre `FrmClientes`.

Processamento: O usuário digita o nome e clica em 'Adicionar'. `FrmClientes` chama `ClienteService.Cadastrar()`.

Persistência: `ClienteService` chama `DataPersistence.Save()`, que sobrescreve `clientes.json`.

Atualização: `FrmClientes` chama `CarregarClientes()`, recarregando o `DataGridView`.

6 REGRAS DE NEGÓCIO IDENTIFICADAS

Regra	Implementação	Funcionamento	Exemplo
Validação de Entrada	Services/*.cs	Impede cadastros com campos nulos/vazios e cadastros duplicados.	Se o argumento Exception sem nome gera erro.
ID Incremental	Services/*.cs	Calcula o próximo ID pelo maior ID existente (Max(cliente)).	Se o ID atual é ID 5 → próximo será 6.
Vínculo Obrigatório	ChamadoService.cs	Chamado não pode ser aberto sem Cliente.	Se o cliente é null ao abrir chamado.
Estado do Chamado	ChamadoService.cs	Novo chamado inicia obrigatoriamente com Aberto.	Novo chamado → Status = Aberto.

7 BANCO DE DADOS (PERSISTÊNCIA)

O sistema utiliza **Persistência em Arquivos Flat (JSON)** em vez de um banco de dados SQL tradicional.

Entidades Armazenadas

- **clientes.json** — Lista de objetos Cliente.
- **atendentes.json** — Lista de objetos Atendente.
- **chamados.json** — Lista de objetos Chamado.

Estratégia de Relacionamento

O relacionamento é implementado via **Composição de Objetos** no JSON. Cada chamado em `chamados.json` contém os objetos completos de Cliente e Atendente **aninhados**, eliminando a necessidade de JOINS.

8 SEGURANÇA

Aspecto	Status	Observação
Autenticação	Inexistente	Qualquer pessoa com acesso ao executável pode gerenciar os dados.
Autorização	Inexistente	Sem níveis de acesso (Admin vs Usuário).
Criptografia	Inexistente	Dados salvos em texto simples (JSON) no diretório bin/Debug.
Vulnerabilidade Principal	Alta	Exposição dos arquivos JSON e falta de validação de permissões.

9 PADRÕES DE PROJETO E MELHORIAS

Padrões de Projeto Utilizados

- **Layered Architecture:** Separação clara entre UI, Negócio e Dados.
- **Singleton-like (Static Services):** Uso de classes estáticas para persistência simplifica o acesso.
- **Data Transfer Object (DTO):** As classes em Models atuam como DTOs para serialização JSON.

Sugestões de Refatoração

Migração para Banco de Dados: Substituir arquivos JSON por SQLite ou SQL Server via Entity Framework Core para suportar mais dados e garantir integridade referencial.

Autenticação: Adicionar tela de Login para proteger os dados dos usuários.

Injeção de Dependência: Substituir instâncias manuais (`new ClienteService()`) por container de DI para facilitar testes unitários.

Validações Avançadas: Implementar validação de e-mail e telefone nos modelos de Cliente.

Paginação: Se a lista de chamados crescer muito, o `DataGridView` ficará lento — implementar paginação ou carregamento sob demanda.

10 RESUMO PARA APRESENTAÇÃO

O que é?

Um sistema de gestão de tickets de suporte desenvolvido em **C#** com **Windows Forms**.

Como funciona?

A aplicação permite cadastrar clientes e técnicos, gerenciando a vida útil de um chamado — desde a abertura até a resolução — salvando todas as informações em arquivos JSON para garantir persistência dos dados.

Tecnologias

.NET 10, C#, WinForms e System.Text.Json.

Problemas Resolvidos

Elimina a desorganização de pedidos de suporte, permitindo que a equipe saiba exatamente quantos chamados estão abertos e quem é o técnico responsável por cada um.

Diferenciais

- Interface moderna com cores customizadas.
- Dashboard de estatísticas em tempo real.
- Formatação visual de status com cores semafóricas (vermelho / laranja / verde).

Analogia Didática: O Caderno de Recados

Imagine que você tem um caderno onde anota quem são seus clientes e quem são seus funcionários. Quando um cliente liga com um problema, você abre uma página nova (o Chamado), escreve o nome do cliente, escolhe qual funcionário vai resolver e descreve o problema. No início, coloca um post-it vermelho 'ABERTO'. Quando o funcionário começa a trabalhar, troca por um amarelo 'EM ANDAMENTO' e, ao final, um verde 'RESOLVIDO'. Este software faz exatamente isso, mas em vez de papel, usa arquivos JSON no computador.

Models → 'folhas do caderno' (definem o que anotar) | Services → 'regras de preenchimento' | Forms → 'capa e páginas' | DataPersistence → 'o armário'